How `src/model.py` works

A walkthrough of the springs-and-sticks PyTorch SDE module

`src/model.py` defines two `nn.Module` classes that together implement the springs-and-sticks (SS) SDE in a form `torchsde.sdeint` can integrate:

- `GroupGS3DE` — a divide-and-conquer wrapper that splits the input features into groups and instantiates one core model per group.
- `GS3DE` — the actual springs-and-sticks SDE on a single (sub-)domain. All of the interesting physics lives here.

Contents

---

1 Architecture: `GroupGS3DE` wraps `GS3DE`

Building one symbolic Lagrangian over a full d-dimensional grid scales as $O(N_s^d)$ degrees of freedom — exponential in the input dimension. `GroupGS3DE` mitigates this by partitioning the d input features into smaller groups (of size $\leq$ `max_features`) and giving each group its own `GS3DE` model.

1. `split_features(num_features, strategy)` returns a list of index tensors, either sequential (`torch.split(arange(d), max_features)`) or random (`torch.split(torch.randperm(d), ...)`).

2. For each index group, a separate `GS3DE` is constructed with the corresponding slice of the boundaries. Construction can run in a `ThreadPoolExecutor` when `parallel=True`.

3. `self.models` is an `nn.ModuleList`; `self.state_sizes` tracks each sub-model's state size; `self.cstate_sizes` is the cumulative sum, used to slice a joint `theta` into per-model chunks.

All glue methods — `f`, `g`, `cost`, `loss`, `y_prediction`, `num_y_prediction` — iterate over `self.models`, call the same method on each, and either concatenate (`f, g`), sum (`cost`), or average (`y_prediction`) the results. `get_theta_group(theta, i)` returns the slice of `theta` that belongs to sub-model i:

```
def get_theta_group(self, theta, group, predict=False):
    if predict:
        return theta[self.cstate_sizes[group]:self.cstate_sizes[group+1]]
    return theta[:, self.cstate_sizes[group]:self.cstate_sizes[group+1]]
```

The rest of this document is about `GS3DE`; `GroupGS3DE` is a thin orchestrator.

2 `GS3DE.__init__`: physical parameters and one-time setup

The constructor stores the physical parameters (`k`, `M`, `friction`, `temp`, `kb`, `k2`) and computes a few derived quantities used everywhere else:

```
self.ell        = (self.u_max - self.u_min) / self.n_sticks
self.M          = M / torch.prod(self.n_sticks)
self.state_size = torch.prod(self.n_sticks + 1) * 2 * n_labels
self.eta_cte    = float(np.sqrt(2 * self.friction * temp * kb / self.M))
```

`ell`	cell width per axis.
`M`	total mass distributed evenly across sticks.
`state_size`	2 phase-space coords (position + velocity) $\times \prod_b (N_s^b + 1)$ grid vertices $\times$ `n_labels` output channels. This is the dimension `torchsde` integrates.
`eta_cte`	the noise amplitude $\sigma = \sqrt{2\gamma T k_B / M}$ from the fluctuation–dissipation relation.

Then it performs the *symbolic* setup, once and for all, in fixed order:

1. `_init_symbols` — create sympy dynamic symbols on the grid.
2. `_init_kinetic_energy` — build $K_{\text{tr}}, K_{\text{rot}}$.
3. `_init_elastic_energy` — build U_{el} (only nonzero if `k2 != 0`).
4. `_init_lagrangian` — assemble L_{nonpot}, derive the mass matrix and non-potential forcing, and call `_update_total_lagrangian` once.

The split: static skeleton vs. data-dependent piece

The data-dependent potential $U(\mathbf{x})$ is **not** built in `__init__`: it is supplied later via `update_data(u_i, y_i)`. Everything that does not depend on the data — the symbolic positions, the kinetic energies, the mass matrix — is computed exactly once. This is the whole point of the `_init_lagrangian` / `_update_total_lagrangian` split.

3 The symbolic skeleton

`_init_symbols`

Builds a numpy array of `sympy.dynamicsymbols` with shape

$$(\underbrace{N_s^0 + 1, \ldots, N_s^{d-1} + 1}_{\text{grid}}, \underbrace{m}_{\text{output channels}}).$$

So `x_symbols[i, j, ..., p]` is the height $x^p_{(i,j,\dots)}$ of the stick at that grid vertex in output channel p. The arrays `dx_symbols` and `ddx_symbols` hold the first and second time-derivatives of each symbol. Total number of symbolic variables: `self.N = prod(shape)`.

`_init_kinetic_energy`

For each axis b it takes the "front" slice (`slice(1, None)`) and "back" slice (`slice(None, -1)`) along that axis to extract neighboring stick velocity pairs $(\dot{x}_{\vec{i}+\hat{e}^b}, \dot{x}_{\vec{i}})$:

```
for i in range(len(shape) - 1):                # over input dims
    sf = [slice(None)]*len(shape); sf[i] = slice(1, None)
    sb = [slice(None)]*len(shape); sb[i] = slice(None, -1)
    diff = (dx_symbols[tuple(sf)] + dx_symbols[tuple(sb)])**2
    ktr += (M/8) * Add(*diff.ravel())          # translational
    diff = (dx_symbols[tuple(sf)] - dx_symbols[tuple(sb)])**2
    krot += (M/24) * Add(*diff.ravel())        # rotational
```

The result is sympy expressions:

$$K_{\text{tr}} = \frac{M}{8}\sum_{b,\vec{i}}(\dot{x}_{\vec{i}+\hat{e}^b} + \dot{x}_{\vec{i}})^2, \qquad K_{\text{rot}} = \frac{M}{24}\sum_{b,\vec{i}}(\dot{x}_{\vec{i}+\hat{e}^b} - \dot{x}_{\vec{i}})^2.$$

`_init_elastic_energy`

Same neighbor pattern but on positions, with an offset by `self.ell[i]`:

$$U_{\text{el}} = \frac{k_2}{2}\sum_{b,\vec{i}}\left(x_{\vec{i}+\hat{e}^b} - x_{\vec{i}} - \ell^b\right)^2.$$

Skipped (left as 0) when `k2 == 0`, which is the default.

`_init_lagrangian`

Combines the non-potential pieces and runs sympy's Euler–Lagrange machinery:

```
self.L_nonpot = simplify(Add(self.ktr, self.krot, -self.uelastic_symbols))
LM = LagrangesMethod(self.L_nonpot, self.x_symbols.flatten())
LM.form_lagranges_equations()
self.mass_matrix     = simplify(LM.mass_matrix)
self.inv_mass_matrix = simplify(LM.mass_matrix.inv())   # falls back to .pinv()
self.forcing_nonpot  = simplify(LM.forcing)
self._update_total_lagrangian()                         # see next section
self.ypred = lambda u: lambdify([*x_symbols.flatten()],
                                self.y_prediction(u))
```

`self.mass_matrix` and `self.inv_mass_matrix` live for the lifetime of the model: they don't depend on data.

4 Bringing in data: `update_data`

`update_data(new_u_i, new_y_i)` does exactly two things:

`_compute_potential_energy(u_i, y_i)`

1. `find_box(u_i)` returns the integer multi-index $\vec{i}$ of the cell that contains $\mathbf{u}_i$.
2. `y_prediction(u_i, i_idx)` returns the symbolic vector $\hat{\mathbf{y}}(\mathbf{u}_i)$ as a multilinear combination of the surrounding stick heights (see Section 5).

3. Builds the symbolic potential

$$U(\mathbf{x}) \;=\; \frac{k}{2}\sum_{j,p}\big(\hat{y}^p(\mathbf{u}_j) - y_j^p\big)^2.$$

`_update_total_lagrangian`

```
self.lagrangian    = Add(self.L_nonpot, -self.U)
self.forcing_pot   = Matrix([-diff(self.U, q) for q in x_symbols.flatten()])
self.forcing_vector = Add(self.forcing_nonpot, self.forcing_pot)
self.evol_dynamics = simplify(self.inv_mass_matrix * self.forcing_vector)

self.lambdified_dyn = lambdify(
    [*x_symbols.flatten(), *dx_symbols.flatten()],
    self.evol_dynamics - self.friction * dx_symbols.reshape(-1, 1))
self.ue = lambdify([*x_symbols.flatten()], self.U)
```

After this call:

- `lambdified_dyn(*q, *dq)` returns $\ddot{\mathbf{x}} = \mathbf{M}^{-1}\mathbf{F}(\mathbf{x},\dot{\mathbf{x}}) - \gamma\dot{\mathbf{x}}$ as fast numerical code (compiled by sympy's `lambdify`).
- `ue(*q)` returns $U(\mathbf{x})$ numerically.

These two compiled functions are what `f`, `cost`, `dw` actually call at integration time — no sympy in the hot loop.

5 Piecewise-linear prediction

Three helpers turn an input $\mathbf{u}$ into a prediction:

```
def find_box(self, u):
    i = ((u - self.u_min) / self.ell).to(torch.int)
    return torch.clip(i, 0, self.n_sticks - 1)

def lamd(self, i, u, flip_ind=None, div_by_ell=True):
    val = u - i*self.ell - self.u_min
    if div_by_ell: val /= self.ell
    return val                                  # the lambda_i^b(u)
```

`find_box` maps $\mathbf{u}$ to the integer cell index $\vec{i}$. `lamd` returns the in-cell coordinate $\lambda_{\vec{i}}^{b}(\mathbf{u}) \in [0,1]$. `_y_prediction(u, i=None)` assembles the multilinear interpolant:

```
u = torch.clamp(u, u_min, u_max)
i = find_box(u) if i is None else i
ld = lamd(i, u)
offsets = torch.tensor(list(product([0,1], repeat=d)))
grid_points = i.unsqueeze(1) + offsets.unsqueeze(0)
weights = torch.prod((1-ld)*(offsets==0) + ld*(offsets!=0), axis=2)
assert torch.all(weights.sum(axis=1) - 1 < 1e-5)
symbols = self.x_symbols[index_tuple]
pred = sum(weights * symbols, axis=corners)
```

The assertion is the sanity check that $\sum_{\vec{j}} w_{\vec{j}} = 1$ inside every cell. The result `pred` is a vector of *symbolic* expressions in the stick positions — ready to be squared into a potential.

`num_y_prediction(u, theta)` is the purely numerical sibling: it evaluates the cached `self.ypred(u)` (which is `lambdifyd` from the symbolic prediction) on the positions stored in `theta`.

6 The SDE interface: `f` and `g`

These are the two methods `torchsde.sdeint` calls at every Euler step. The argument `theta` has shape `(batch, state_size)` with the first half holding positions $\mathbf{x}$ and the second half velocities $\dot{\mathbf{x}}$.

`f(t, theta)` — the drift

```python
def f(self, t, theta):
    q  = theta[:, :self.N].t()                  # positions
    dq = theta[:, self.N:].t()                  # velocities
    ddqdt = torch.tensor(self.lambdified_dyn(*q, *dq))
    return torch.vstack([dq, ddqdt]).t()
```

This computes

$$\mathbf{f}(\boldsymbol{\theta}) \;=\; \begin{pmatrix} \dot{\mathbf{x}} \\ \mathbf{M}^{-1}\mathbf{F}(\mathbf{x},\dot{\mathbf{x}}) - \gamma\dot{\mathbf{x}} \end{pmatrix}.$$

The branch on `ddqdt_shape` handles the degenerate one-dimensional case.

`g(t, theta)` — the diffusion

```python
def g(self, t, theta):
    return torch.cat([
        torch.zeros_like(theta[:, :self.N]),
        self.eta_cte * torch.ones_like(theta[:, self.N:])
    ], dim=1)
```

Position rows are zero (positions inherit noise only *indirectly*, through integrating noisy velocities); velocity rows are `eta_cte` $= \sigma$. Combined with `noise_type = "diagonal"` and `sde_type = "ito"`, this tells `torchsde` the SDE is

$$\mathrm{d}\boldsymbol{\theta} \;=\; \mathbf{f}(\boldsymbol{\theta})\,\mathrm{d}t \;+\; \mathbf{B}\,\mathrm{d}W_t, \qquad \mathbf{B} = \mathrm{diag}(0,\ldots,0,\sigma,\ldots,\sigma).$$

7 Loss, energy, work

`cost(theta)`	Evaluates `self.ue(*q)` — the data potential $U(\mathbf{x})$ on each batch element. Since $U = (k/2)\sum(\hat{y}-y)^2$, this is the (unnormalized) squared-error loss times $k/2$.
`loss(theta, u_i, y_i)`	Returns $2U/(N\cdot k)$ = MSE per data point.
`dcost(theta)`	The total time-derivative $\mathrm{d}U/\mathrm{d}t$, computed by sympy via `self.U.diff()` and then `lambdifyd`.
`dw(theta)`	The instantaneous power $\mathrm{d}W/\mathrm{d}t = M\langle\ddot{\mathbf{x}},\dot{\mathbf{x}}\rangle$ — used by the Jarzynski free-energy estimator in `src/entropy.py`.

```python
def dw(self, theta):
    q  = theta[:, :self.N].t()
    dq = theta[:, self.N:].t()
    ddqdt = torch.tensor(self.lambdified_dyn(*q, *dq)).squeeze()
    return torch.sum(ddqdt * dq, dim=0) * self.M
```

8 Lifecycle in a training loop

The usage pattern (as in `src/profiling.py`) is:

```
sde = GroupGS3DE(max_features, n_sticks, boundaries, n_labels, ...)
theta0 = None
for u_i, y_i in loader_train:
    sde.update_data(u_i, y_i)                # rebuild U, ddqdt, ue
    if theta0 is None:
        theta0 = torch.rand((num_runs, sde.state_size))
    thetas = torchsde.sdeint(sde, theta0, ts, method='euler')
    # sde.f and sde.g are called at every Euler step
    theta0 = thetas[-1, :]                   # warm-start next batch
```

Only data-dependent pieces are rebuilt between mini-batches. The kinetic energy, mass matrix, and elastic energy are computed exactly once in `__init__`; the data potential and the compiled drift function are rebuilt on every `update_data`; the SDE solver only ever calls the cheap numerical `f, g`.

9 How are the sticks actually moved forward in time?

There is a tiny time increment Δt and the integration scheme is **Euler–Maruyama**, but `model.py` does not run the time-stepping itself — it just hands `f` and `g` to `torchsde.sdeint`, which loops in C.

The state vector is

$$\boldsymbol{\theta}(t) \;=\; \begin{pmatrix} \mathbf{x}(t) \\ \dot{\mathbf{x}}(t) \end{pmatrix} \in \mathbb{R}^{2N}.$$

One Euler–Maruyama step is

$$\boldsymbol{\theta}(t+\Delta t) \;=\; \boldsymbol{\theta}(t) \;+\; \mathbf{f}(t,\boldsymbol{\theta})\,\Delta t \;+\; \mathbf{g}(t,\boldsymbol{\theta})\,\sqrt{\Delta t}\,\boldsymbol{\eta}, \qquad \boldsymbol{\eta} \sim \mathcal{N}(0,\mathbb{I}).$$

Concretely, what `torchsde.sdeint(sde, theta0, ts, method='euler')` does is essentially:

```
theta = theta0
for k in range(len(ts) - 1):
    dt    = ts[k+1] - ts[k]
    eta   = torch.randn_like(theta)
    drift = sde.f(ts[k], theta)              # (dx/dt, d2x/dt2)
    noise = sde.g(ts[k], theta)              # (0,...,0, sigma,...,sigma)
    theta = theta + drift*dt + noise*eta*sqrt(dt)
```

So `sde.f` returns *velocities and accelerations* (how fast positions and velocities are currently changing), and the solver multiplies that by `dt` to push the state forward one tick. The accelerations themselves are computed inside `sde.f` by calling `self.lambdified_dyn(*q, *dq)`, which is a numerically compiled function that evaluates

$$\ddot{\mathbf{x}} \;=\; \mathbf{M}^{-1}\mathbf{F}(\mathbf{x},\dot{\mathbf{x}}) - \gamma\dot{\mathbf{x}}$$

at the current state.

Where does Δt come from?

Δt comes entirely from the user's choice of `ts = torch.linspace(t_start, t_end, t_size)`; the solver uses the gaps between successive entries of `ts`. In `profiling.py` you have `ts = torch.linspace(0, 1, 2)`, which means one Euler step of size $\Delta t = 1$ (good for profiling, bad for accuracy). For real runs `t_size` should be in the hundreds or thousands so Δt is small.

Take-away. The sticks "move" inside `torchsde.sdeint`, not in any `model.py` function. `model.py` only specifies the *rules* of motion (`f` for the drift, `g` for the noise); the solver applies those rules step by step.

10 Function-wise flow at a glance

```
profiling.run_main()
|
+-- GroupGS3DE.__init__()                          [ONCE, ~seconds-minutes]
|   +-- GS3DE.__init__()         per feature group
|       +-- _init_symbols()                    <- sympy variables
|       +-- _init_kinetic_energy()             <- K_tr, K_rot
|       +-- _init_elastic_energy()             <- U_el (often 0)
|       +-- _init_lagrangian()
|           +-- LagrangesMethod(...)           <- mass_matrix, inv_mass_matrix
|           +-- _update_total_lagrangian()     <- no-op: U is still 0
|
+-- for each epoch:
    +-- for each mini-batch (u_i, y_i):
        |
        +-- sde.update_data(u_i, y_i)                [PER BATCH]
        |   +-- _compute_potential_energy(u_i, y_i)
        |   |   +-- find_box(u_i)                    <- which cell?
        |   |   +-- y_prediction(u_i, i_idx)         <- symbolic y_hat
        |   |       +-- _y_prediction()
        |   |           +-- lamd(i, u)               <- lambda in [0,1]
        |   |           +-- (multilinear corner weights)
        |   +-- _update_total_lagrangian()
        |       +-- lagrangian   = L_nonpot - U
        |       +-- evol_dynamics = M^-1 * forcing
        |       +-- lambdified_dyn = lambdify(...)   <- FAST numeric ddx(x,dx)
        |
        +-- sde.loss(theta0, u_i, y_i)               <- report MSE
        |
        +-- torchsde.sdeint(sde, theta0, ts, 'euler') [PER EULER STEP x t_size]
        |   +-- for k in range(t_size - 1):
        |       +-- sde.f(t, theta)
        |       |   +-- lambdified_dyn(*q, *dq)      <- compute ddx
        |       +-- sde.g(t, theta)                  <- sigma on velocity rows
        |       +-- theta += f*dt + g*eta*sqrt(dt)   <- Euler-Maruyama
        |
        +-- theta0 = thetas[-1]                      <- warm-start next batch
```

How to read this tree

- **Top block (`__init__`)** runs once and builds the symbolic skeleton that does *not* depend on data: mass matrix, inverse mass matrix, kinetic energies. This is the slow sympy work; it is amortized over the whole training run.
- `update_data` **(per batch)** rebuilds only the data-dependent piece $U(\mathbf{x})$ and recompiles the numerical drift function `lambdified_dyn`. The mass matrix is *not* rebuilt — it never changes.
- `sdeint` **(per Euler step inside a batch)** is the hot loop. It only ever calls the cheap numerical functions `f` and `g` — no sympy in this loop. Each call advances the state by one Δt.

11 Summary table

Method	Role	Recomputed
`_init_symbols`	sympy variables on the grid	once
`_init_kinetic_energy`	$K_{\mathrm{tr}}, K_{\mathrm{rot}}$ symbolically	once
`_init_elastic_energy`	U_{el} symbolically (if `k2`$\neq 0$)	once
`_init_lagrangian`	L_{nonpot}, mass matrix, non-pot. forcing	once
`_compute_potential_energy`	build $U(\mathbf{x})$ from new data	per `update_data`
`_update_total_lagrangian`	rederive $\ddot{\mathbf{x}}$ symbolically, lambdify it	per `update_data`
`find_box, lamd, _y_prediction`	piecewise-linear interpolation kernel	per call
`f(t, theta)`	SDE drift for `torchsde`	per Euler step
`g(t, theta)`	SDE noise matrix	per Euler step
`cost, loss, dcost, dw`	diagnostics (energy, MSE, power)	on demand